

Sistemas operativos

Proyecto 3: Comandos EAFITos – Lazy Allocation

MBA, I.S. José Luis Montoya Pareja
Departamento de Ingeniería de Sistemas
Universidad EAFIT
Medellín, Colombia, Suramérica

Resumen

Este proyecto académico propone el desarrollo de EAFITos, una shell educativa implementada en lenguaje C, orientada a reforzar conceptos fundamentales de Los Sistemas Operativos mediante una experiencia práctica y progresiva. El proyecto se enmarca en un curso universitario de Sistemas Operativos que toma como referencia la filosofía educativa de xv6, un sistema operativo desarrollado por el MIT para la enseñanza, caracterizado por su simplicidad, claridad estructural y cercanía al funcionamiento real de un sistema operativo. A través del diseño e implementación de una shell interactiva con comandos básicos y avanzados, los estudiantes exploran aspectos clave como la interacción usuario-sistema, el manejo de archivos, la gestión de procesos, el uso de llamadas al sistema y las buenas prácticas de programación en C. El proyecto puede desarrollarse de manera individual o colaborativa, promoviendo además competencias transversales como el trabajo en equipo, el control de versiones y la documentación técnica. Como resultado final, se espera una aplicación funcional, modular y documentada, que evidencie comprensión conceptual y capacidad de aplicar principios de Sistemas Operativos en un contexto práctico relacionado con la gestión de memoria.

Palabras clave

Sistemas Operativos, xv6, shell, programación en C, interacción usuario-sistema, memoria.

Contexto

El proyecto se desarrolla en el marco del curso de Sistemas Operativos, cuyo objetivo es que los estudiantes comprendan tanto los fundamentos teóricos como su aplicación práctica. En este contexto, xv6 se utiliza como referencia pedagógica por su diseño minimalista y su alineación con

los principios clásicos de los sistemas tipo Unix, lo que facilita el aprendizaje progresivo de conceptos complejos.

La implementación de una shell educativa para EAFITos permite a los estudiantes experimentar de manera directa cómo los sistemas operativos ofrecen servicios al usuario, cómo se estructuran las interfaces de línea de comandos y cómo se gestionan recursos como archivos y procesos desde el espacio de usuario. El proyecto busca desarrollar competencias técnicas en programación en C, diseño modular y manejo de errores, así como habilidades de trabajo colaborativo, uso de herramientas de control de versiones y documentación técnica, todas esenciales en la formación de un ingeniero de sistemas.

Objetivos de aprendizaje

Agregar a la shell educativa de EafitOS en lenguaje C comandos de gestión de memoria que permita aplicar y consolidar conceptos fundamentales de Sistemas Operativos, en coherencia con la filosofía educativa de xv6.

Objetivos específicos

- Comprender el camino completo de una llamada al sistema: user → ecall → trap → dispatcher → sys_ → retorno a user*.
- Ubicar físicamente las piezas clave en el árbol de xv6.
- Construir un trazador de syscalls que conecte lo que ocurre en kernel con el comportamiento observable en user space.
- Entender cómo el kernel obtiene argumentos desde el espacio de usuario de manera segura.
- Relacionar el layout virtual del proceso con entradas reales de la tabla de páginas.
- Distinguir regiones por permisos y propósito.
- Ver cómo un permiso de PTE incorrecto (o una escritura ilegal) genera un trap y cómo xv6 lo maneja.
- Practicar el uso de mappages y la interpretación de scause.
- Verificar que el kernel maneja correctamente punteros inválidos y retorna errores (sin panic).
- Relacionar el contenido de trapframe con la decodificación de argumentos en el dispatcher.

Descripción de los comandos a implementar

1. Observando y comprendiendo los Page Faults

Comprender cuándo y por qué ocurre un page fault, y cómo el kernel responde.

- a. Modificar kernel/trap.c:
En usertrap(), detectar:
 - i. Load page fault
 - ii. Store page fault
- b. Imprimir:
 - i. PID
 - ii. scause
 - iii. stval (dirección virtual que falló)
- c. Crear programa user/tpf.c:
 - i. Intentar leer/escribir en direcciones no mapeadas
 - ii. Ejemplo: punteros arbitrarios (0x0, 0x4000, etc.)
- d. Documentar y observar:
 - i. Qué tipo de fault ocurre en cada caso

Resultado esperado

- Mensajes en kernel indicando:

page fault: pid=3 scause=15 stval=0x4000
- El proceso con el page fault es terminado por acceso inválido.

2. Transformando sbrk(): introducción a Lazy Allocation

Comprender la diferencia entre:

- **Asignación inmediata (actual)**
 - **Asignación diferida (lazy)**
- a. Modificar sys_sbrk() en kernel/sysproc.c:
 - i. **Eliminar la asignación física inmediata**
 - ii. Solo aumento sz del proceso
 - b. En proc.c:
 - i. Asegurarse de que el proceso tenga el nuevo tamaño virtual
 - c. Ejecutar un programa que:
 - i. Llame sbrk(4096)
 - ii. Acceda a esa memoria

Resultado esperado

- El programa falla con page fault al acceder a memoria recién reservada.

3. Implementando Lazy Allocation real

Implementar lazy allocation funcional usando page faults.

- a. En usertrap() (kernel/trap.c):
 - i. Si detecta page fault:
 1. Verifica si stval está dentro de p->sz
 2. Si está:
 - a. Asignar nueva página (kalloc)
 - b. Mapearla con mappages
 3. Si no:
 - a. matar el proceso
 - b. Asegurar:
 - i. Página inicializada en 0
 - c. Probar con programa:
 - i. Reserva memoria con sbrk
 - ii. Accede progresivamente

Resultado esperado

- Ya no hay crash
- La memoria se asigna solo cuando se accede
- Se observan múltiples page faults controlados

4. Control y visualización del comportamiento lazy

Analizar el comportamiento dinámico del lazy allocation.

- a. Instrumentar:
 - i. Contador de page faults por proceso (struct proc)
- b. Modificar:
 - i. Incrementar contador en cada fault válido
- c. Crear programa user/tlazy.c:
 - i. Reserva varias páginas con sbrk
 - ii. Accede:
 1. Secuencialmente
 2. Aleatoriamente
- d. Imprimir:
 - i. Número de page faults.

Resultado esperado

- Diferencias claras:
 - Acceso secuencial → menos faults
 - Acceso disperso → más faults.

5. De Lazy Allocation a Memory Mapped Files (simulación)

Entender cómo lazy allocation es la base de mmap.

- a. Simular un mmap sencillo:
 - i. Definir estructura en proc:

```
struct vregion {
```

- ```
uint64 start;
int size;
}
```
- b. Crear syscall mapzero(size):
    - i. Reserva rango virtual sin mapear
  - c. En usertrap():
    - i. Si fault está en esa región:
      1. Asignar página
      2. Inicializar con patrón (ej: 'A')
  - d. Programa user/tmmap\_sim.c:
    - i. Usa mapzero
    - ii. Accede gradualmente
    - iii. Observa contenido

#### Resultado esperado

- Memoria cargada bajo demanda
- Similar a cómo funciona mmap real.

## Entregable final

Se mantiene la estructura de arquitectura recomendada del proyecto

1. Se agrega una carpeta dentro de src/utils llamada memory donde se crean los comandos nuevos.

## Rúbrica

Funcionalidad (40 puntos)

Implementación de todos los comandos (8 puntos por comando)

Calidad del código (30 puntos)

| Criterio      | Puntaje | Descripción           |
|---------------|---------|-----------------------|
| Modularidad   | 8       | Separación en módulos |
| Documentación | 8       | Comentarios + docs    |
| Estilo        | 5       | Convenciones          |
| Memoria       | 4       | Sin leaks             |
| Makefile      | 5       | Build system          |

Colaboración (20 puntos - solo equipos)

| Criterio     | Puntaje | Descripción         |
|--------------|---------|---------------------|
| Git usage    | 5       | Commits, branches   |
| Distribución | 5       | Trabajo equilibrado |
| Integración  | 5       | Código integrado    |
| Roles        | 5       | Roles definidos     |

### Features Avanzadas (10 puntos)

| Criterio            | Puntos | Descripción                    |
|---------------------|--------|--------------------------------|
| Pruebas automáticas | 3      | Unit tests                     |
| Sistema de ayuda    | 2      | Ayuda por comando              |
| Manejo de señales   | 2      | Ctrl+C, Ctrl+Z                 |
| Interfaz mejorada   | 3      | Colores, prompt personalizable |

Nota máxima: 100 puntos (5.0)

## Fecha de entrega

Martes 12 de mayo.

## Referencias

MIT. (2023). A simple, Unix-like teaching operating system.  
<https://pdos.csail.mit.edu/6.828/2025/xv6.html>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). Operating System Concepts (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson Education.

Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press.